

PDF Mismatch Detection Agent

Overview

This project aims to build a modular AI-assisted document comparison system capable of identifying mismatches between two PDF documents. Unlike traditional text diff tools, this system is designed to compare not only textual content but also document structure, formatting, OCR output, embedded images, and scanned documents.

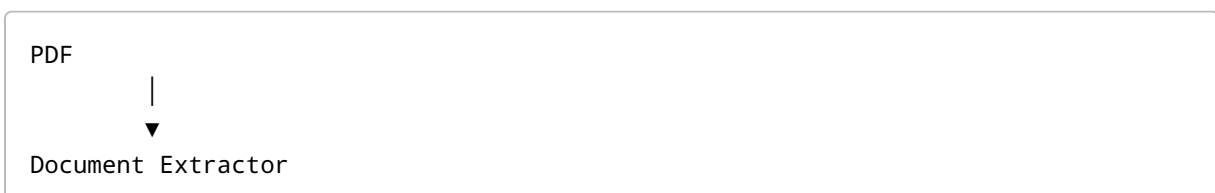
The long-term objective is to create a reusable document analysis framework where different extraction and comparison modules can be plugged into the same comparison pipeline.

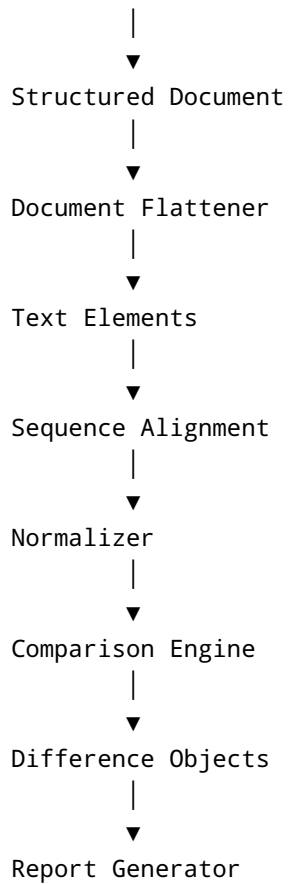
Objectives

The system should eventually detect:

- Character-level differences
 - Word-level differences
 - Sentence insertion/deletion
 - Spelling mistakes
 - Hidden Unicode characters
 - Whitespace differences
 - Formatting differences
 - Font changes
 - Font size differences
 - Font color differences
 - Alignment differences
 - Margin and layout differences
 - Table mismatches
 - Image differences
 - Text inside embedded images
 - OCR differences in scanned PDFs
 - Handwritten document differences (future scope)
-

High-Level Architecture





Each stage has a single responsibility and is independent from the others.

Project Structure

```
pdf_mismatch/  
  
├─ comparators/  
│   └─ text.py  
│  
├─ data/  
│   ├── document_A.pdf  
│   ├── document_B.pdf  
│   ├── document_C.pdf  
│   └─ document_D.pdf  
│  
├─ extractors/  
│   └─ digital.py  
│
```

```
├── models/
│   ├── document.py
│   ├── page.py
│   ├── text_block.py
│   ├── line.py
│   ├── span.py
│   └── text_element.py
├── utils/
│   └── flattener.py
├── main.py
└── requirements.txt
```

Data Model

The extractor converts every PDF into a structured representation.

```
Document
├── Page
│   ├── TextBlock
│   │   ├── Line
│   │   │   └── Span
```

This closely mirrors the hierarchy returned by PyMuPDF.

Span

Smallest formatted text unit.

Stores:

- text
- font
- font size
- color
- bounding box

Line

Collection of spans.

Provides reconstructed line text.

Stores:

- bbox
- spans

TextBlock

Logical paragraph or region.

Stores:

- block number
- bbox
- lines

Provides reconstructed block text.

Page

Stores

- page number
- dimensions
- text blocks

Document

Root object representing the parsed PDF.

Flattened Representation

After extraction the document is transformed into:

TextElement[]

Each TextElement contains

- unique id
- page number
- block number
- line number
- original text
- normalized text (future)
- bounding box

This linear representation becomes the canonical comparison unit.

Why Flatten?

A tree structure is ideal for extraction but inefficient for comparison.

Flattening converts

Document

into

TextElement[]

which allows sequence alignment and diff algorithms to work efficiently while preserving positional metadata.

Planned Modules

Phase 1

Digital PDF extraction

Status: Complete

Phase 2

Sequence Alignment

Responsibilities

- Match corresponding text lines
 - Handle insertions
 - Handle deletions
 - Preserve ordering
-

Phase 3

Normalization

Responsibilities

- Unicode normalization
- Hidden character detection
- Whitespace normalization
- Case normalization
- Smart quote normalization

Normalization will never overwrite original text.

Phase 4

Text Comparator

Will classify differences into:

- Character
 - Word
 - Sentence
 - Case
 - Whitespace
 - Spelling
 - Insertion
 - Deletion
-

Phase 5

Formatting Comparator

Will compare

- Font family
 - Font size
 - Bold
 - Italic
 - Underline
 - Font color
 - Paragraph spacing
 - Alignment
-

Phase 6

Image Comparator

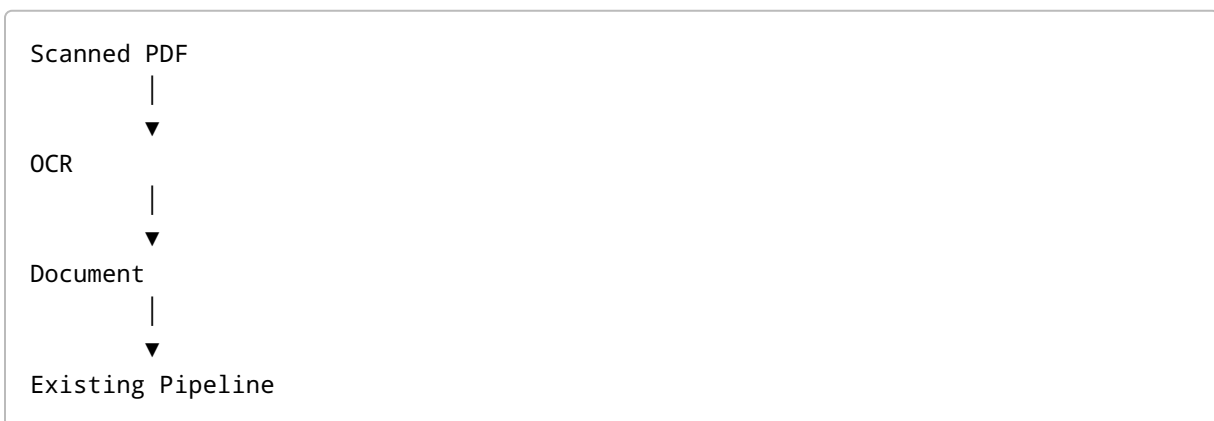
Will compare

- Missing images
 - Modified images
 - OCR text inside images
-

Phase 7

Scanned PDF Support

Pipeline



The comparison engine will remain unchanged because OCR output is converted into the same Document structure.

Phase 8

Report Generator

Outputs

- Console report
 - JSON
 - HTML
 - PDF summary
 - Visual difference highlighting
-

Design Principles

- Single Responsibility Principle
 - Modular architecture
 - Immutable extraction output
 - Separation between extraction and comparison
 - Extensible pipeline
 - Reusable comparison engine
 - Independent modules
-

Long-Term Goal

Create a production-ready document comparison framework capable of comparing both digital and scanned documents while providing detailed, explainable mismatch reports.